

Arbeitsblatt 5: Analogwerte verstehen – Poti, Serial Plotter & map()

Vom Rohwert 0–1023 zur Anwendung: LED dimmen und Servo steuern — 2. Juli 2026

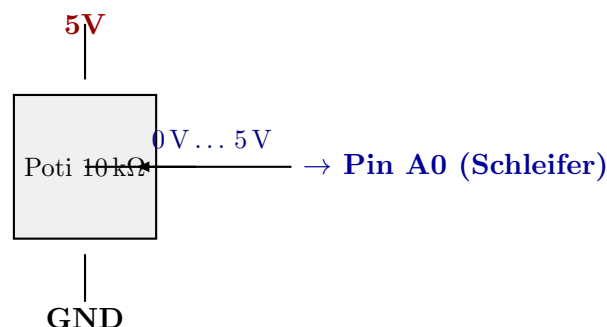
Rückblick: Was bedeuten die Zahlen 0–1023?

Beim NTC (AB 2) hat `analogRead()` Werte zwischen 0 und 1023 geliefert – aber die Änderung war klein und schwer zu “sehen”. Heute nehmen wir ein Bauteil mit dem **vollen Hub**: das **Potentiometer (Poti)**. Damit wird klar, was ein Analogwert ist und wie man ihn mit `map()` in eine **Anwendung** umrechnet.

NTC (AB 2)	Poti (heute)
kleine Änderung, schwer sichtbar	voller Bereich 0 ... 1023 durch Drehen
Temperatur (unsichtbar)	Drehwinkel (sichtbar, direkt)

Teil 1 – Das Potentiometer als einstellbarer Spannungsteiler

Ein Poti hat **3 Anschlüsse**: außen 5V und GND, in der Mitte der **Schleifer**. Beim Drehen verschiebt sich der Abgriff – die Spannung am Schleifer geht stufenlos von 0 V bis 5 V.



Teil 2 – Analogwert “sehen”: der Serial Plotter

Statt nur Zahlen zu lesen, zeichnen wir den Wert als **Kurve**: *Werkzeuge* → *Serieller Plotter*.

```
int potiPin = A0;
int wert;

void setup() {
  Serial.begin(9600);
}

void loop() {
  wert = analogRead(potiPin); // 0 ... 1023
  Serial.println(wert);       // eine Zahl pro Zeile -> Plotter
                               zeichnet
  delay(50);
}
```

1. Baue die Poti-Schaltung auf und lade den Sketch. Öffne den **Seriellen Plotter** (Baudrate 9600).
2. Drehe langsam von einem Anschlag zum anderen. Notiere: kleinster Wert _____, größter Wert _____.
3. Rechne den Rohwert in eine **Spannung** um (das macht die Zahl “lesbar”):

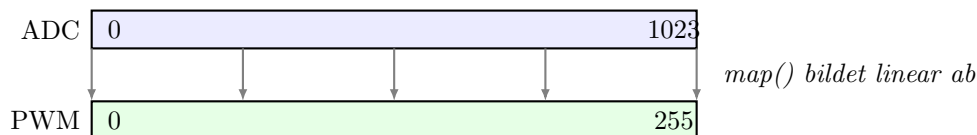
$$U = \text{wert} \cdot \frac{5\text{ V}}{1023} \quad \Rightarrow \quad \text{wert} = 512 \hat{=} \text{_____ V}$$

Teil 3 – map(): Wertebereiche umrechnen

Der Poti liefert **0–1023**. Für Anwendungen brauchen wir oft **andere Bereiche** – z. B. 0–255 (LED-Helligkeit) oder 0–180 (Servo-Winkel). Das erledigt `map()`:

`map(wert, 0, 1023, 0, 255)`

(Eingangswert, von, bis, neu-von, neu-bis)



ADC-Wert	map(...,0,1023,0,255)
0	0
512	
1023	255

Teil 4 – Anwendung A: LED dimmen (PWM)

Verbinde eine **LED** über $220\ \Omega$ mit einem **PWM-Pin** (mit ~, z. B. Pin 9). `analogWrite()` gibt Werte **0–255** aus (0 = aus, 255 = voll).

```
int potiPin = A0;
int ledPin  = 9;      // PWM-Pin (~)
int wert, helligkeit;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  wert = analogRead(potiPin);           // 0 ... 1023
  helligkeit = map(wert, 0, 1023, 0, ____); // <-- ergaenzen
  analogWrite(ledPin, helligkeit);      // LED-Helligkeit setzen
  delay(20);
}
```

4. Ergänze die Lücke. Drehe den Poti – was passiert mit der LED? _____

Teil 5 – Anwendung B: Servo steuern

Ein Servo fährt einen Winkel 0–180° an. Signal an Pin 9, rot an 5V, braun/schwarz an GND.

```
#include <Servo.h>
Servo meinServo;
int potiPin = A0;
int wert, winkel;

void setup() {
    meinServo.attach(9);           // Servo an Pin 9
}

void loop() {
    wert = analogRead(potiPin);    // 0 ... 1023
    winkel = map(wert, 0, 1023, 0, ____); // <-- Winkelbereich
    ergaenzen
    meinServo.write(winkel);       // Winkel anfahren
    delay(20);
}
```

5. Ergänze den Winkelbereich. Beschreibe den Zusammenhang Poti-Stellung → Servo-Winkel:
-

Teil 6 – Digitalisierung & Auflösung (Transfer zu AB 4)

6. Drehe den Poti im Plotter *ganz langsam*. Die Kurve steigt nicht glatt, sondern in **Stufen**. Warum? _____
7. Der ADC hat **10 Bit** = 2^{10} = 1024 Stufen bei 0–5 V. Eine Stufe entspricht

$$\Delta U = \frac{5 \text{ V}}{1024} = \text{_____ mV.}$$

Kleinere Spannungsunterschiede kann der Arduino **nicht** unterscheiden.

8. **Zusatz:** Ersetze `Serial.println(wert)` durch die Spannung (`float U = wert * 5.0 / 1023;`) und gib U aus.

Benötigte Bauteile

- 1× Potentiometer 10 kΩ
- 1× LED, 1× Widerstand 220 Ω
- 1× Servo (z. B. SG90), Steckbrücken



Lösung: <https://hbraak.github.io/fachpraxis/arduino/lsg5.html>